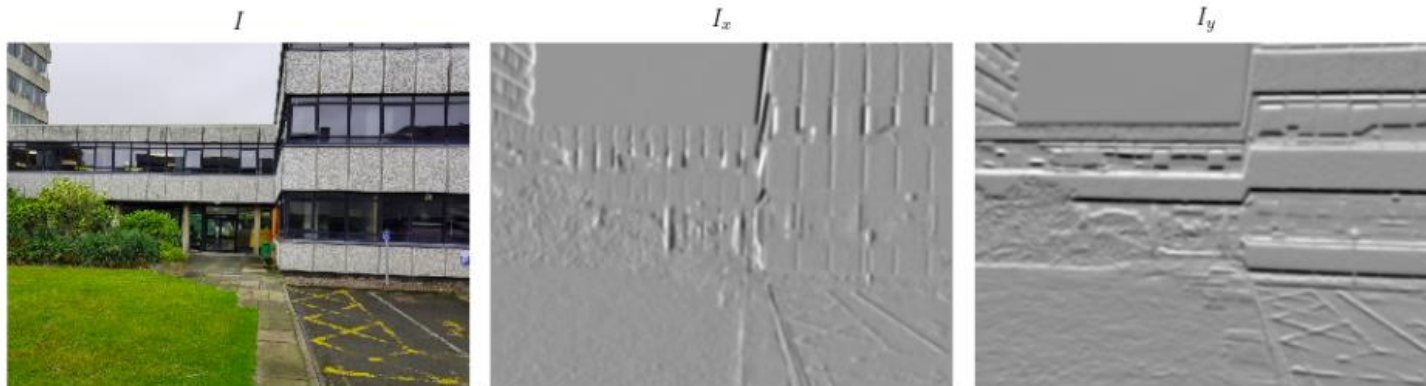


# Feature extraction: Parametric Hough Transform

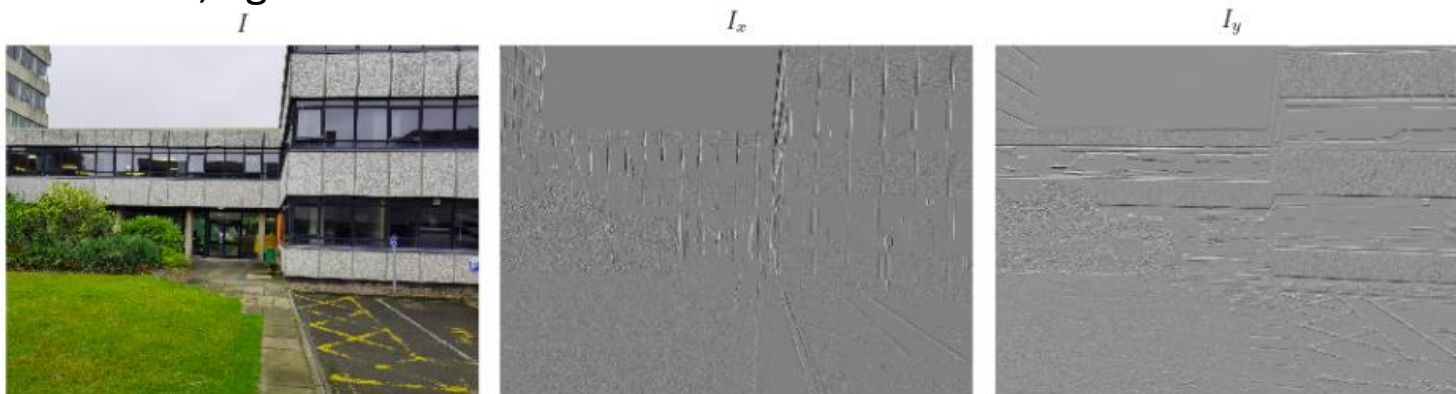
Prof. Jungong Han (jungong.han@sheffield.ac.uk)  
Department of Computer Science  
The University of Sheffield

# Test

I was trying to produce the following results



However, I got these



Problems?

# Outline

- Motivation
- Hough transform: Lines
- Hough transform: Circles

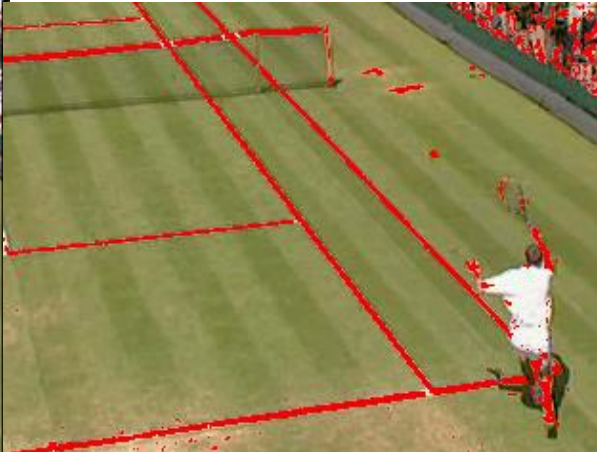


# Image processing vs. CV

- Image processing:  $\hat{I} = F(I)$ , where  $F$  is a set of linear/nonlinear operators applied to the images; the outputs are still **2D images**.
- In CV, we are going to extract both low- and high-level features:

$$F(I) \rightarrow \{images, lines, circles, blobs, corners, etc \dots\}$$

# Line and circle detection



# Parametric models

- Parametric modeling is creating a class of instances to describe a model/data, where each instance is represented by a set of parameters.

- Example:

$$y = m x + b$$

instances : lines

parameters:  $\{m, b\}$

- Models could be of higher orders like circles, ellipses, and any arbitrary shapes

# Line model

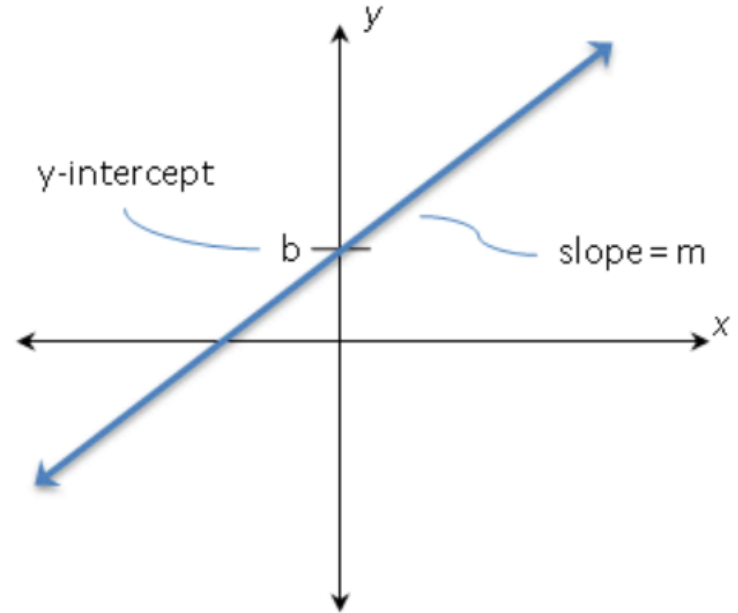
- A straight line is defined as

$$y = m x + b$$

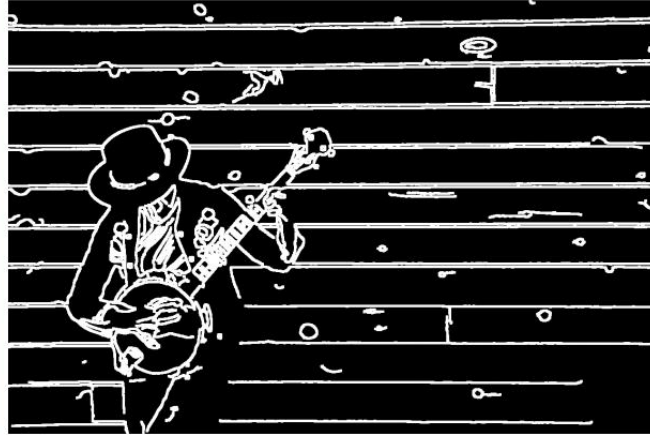
*(Red arrow points from  $m$  to "slope of the line")*  
*(Red arrow points from  $b$  to "y-intercept")*

slope of the line

y-intercept



# Aren't lines (edges) enough?

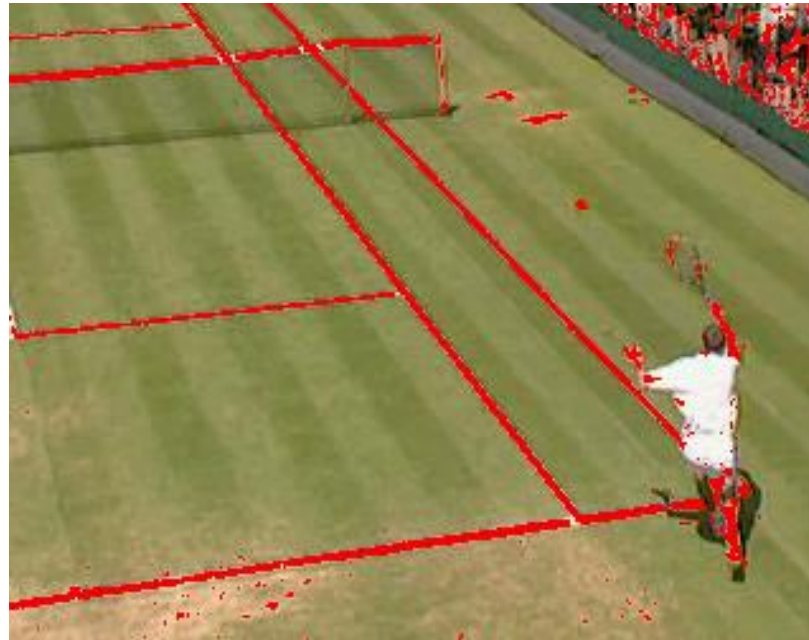


- ✓ Detecting the lines (with Canny edge detector)
- ✓ Noise & distortion across the edges
- ✓ Many edges unrelated to lines
- ✓ Lines are partially detected



# Considerations

- ✓ Given edge pixels, how can we get the line parameters?

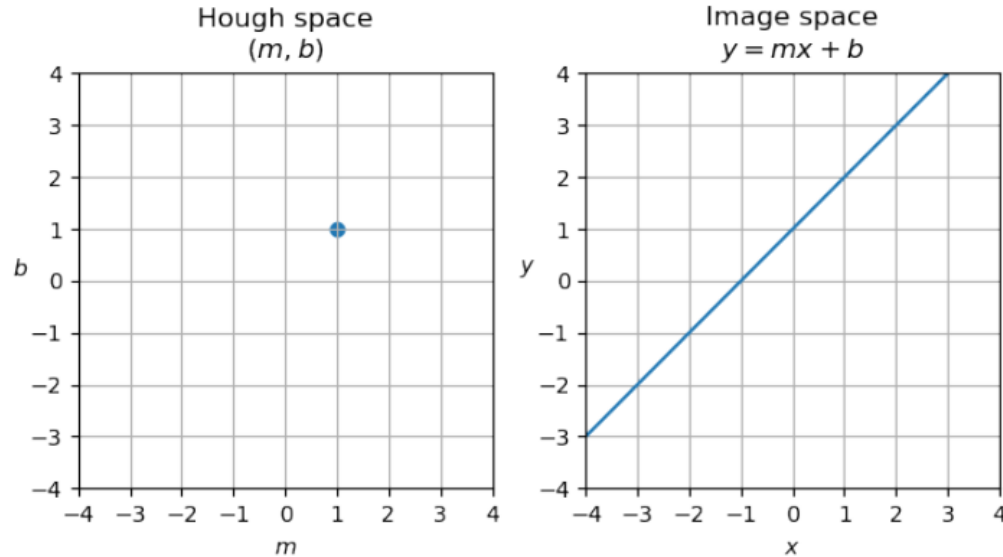


# Hough transform

- Proposed by: Hough, 1962
- Firstly used to detect lines in images by: Duda, 1972
- Very simple idea:
  - Each edge coordinate votes for all lines that it could be part of
  - Select most voted lines
- It is still widely used in image processing and CV!
  - Xue et al., 'Learning Regional Attraction for Line Segment Detection', IEEE Transactions on Pattern Analysis and Machine Intelligence, 2021

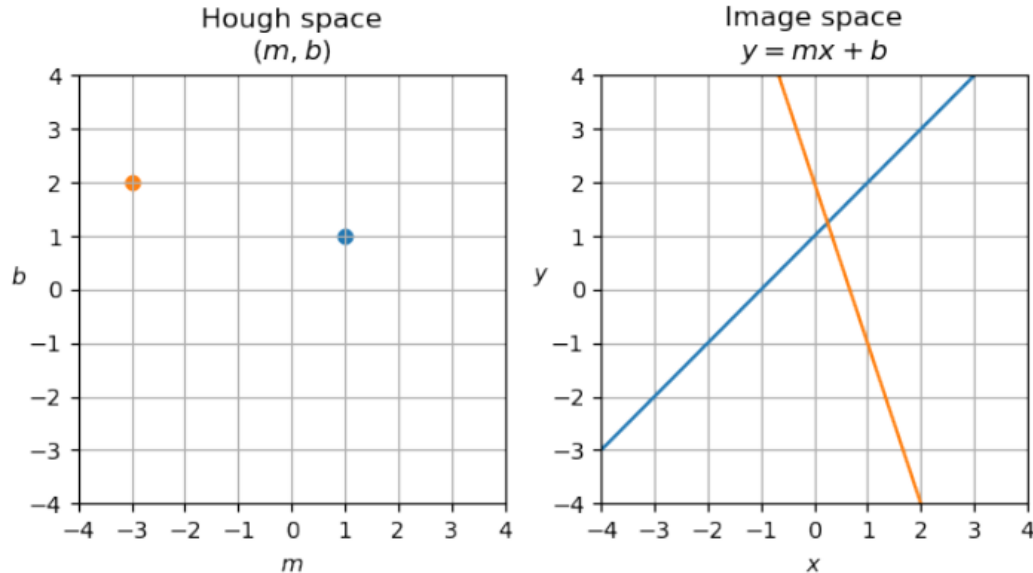
# Hough space

- Each pair of parameters (point) in the Hough space defines 1 line in the image space



# Hough space

- Each pair of parameters (point) in the Hough space defines 1 line in the image space



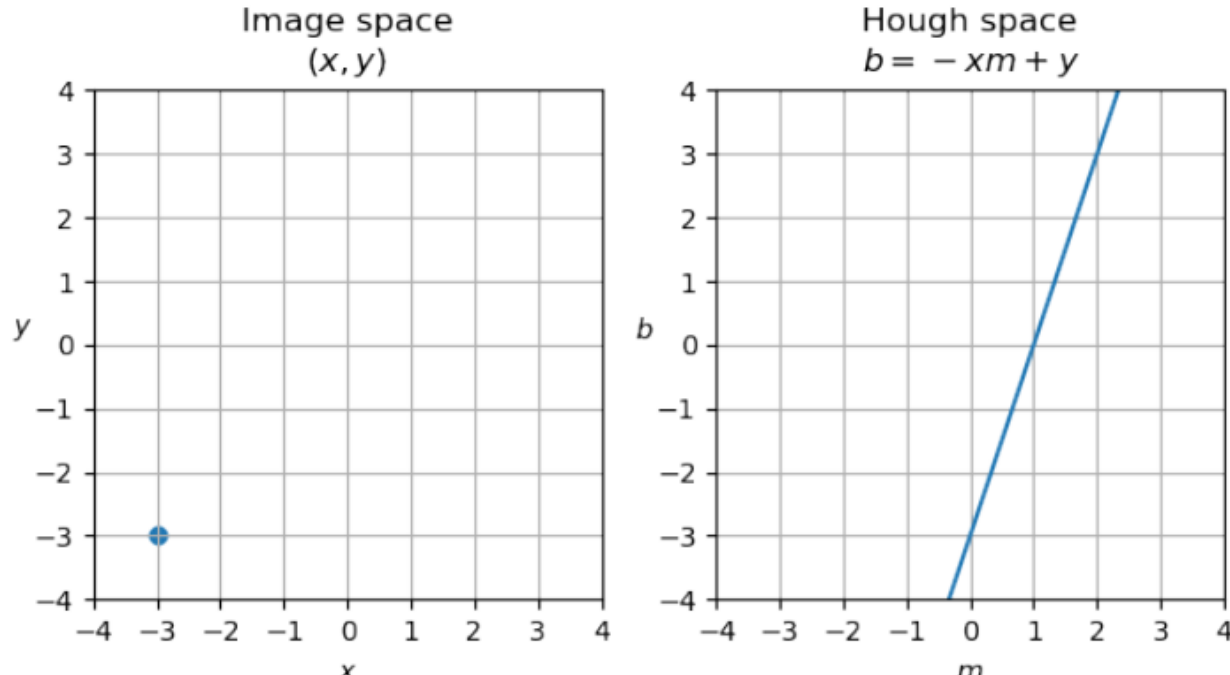
# 1 point in image space

- What if we have a point and we want to know what are the possible lines that the point belongs to?
- That, is, given an  $(x, y)$ , what are the possible  $(m, b)$ ?
- We need to solve the following equation with respect to  $b$  (or  $m$ ):

$$y = mx + b \Leftrightarrow b = -mx + y$$

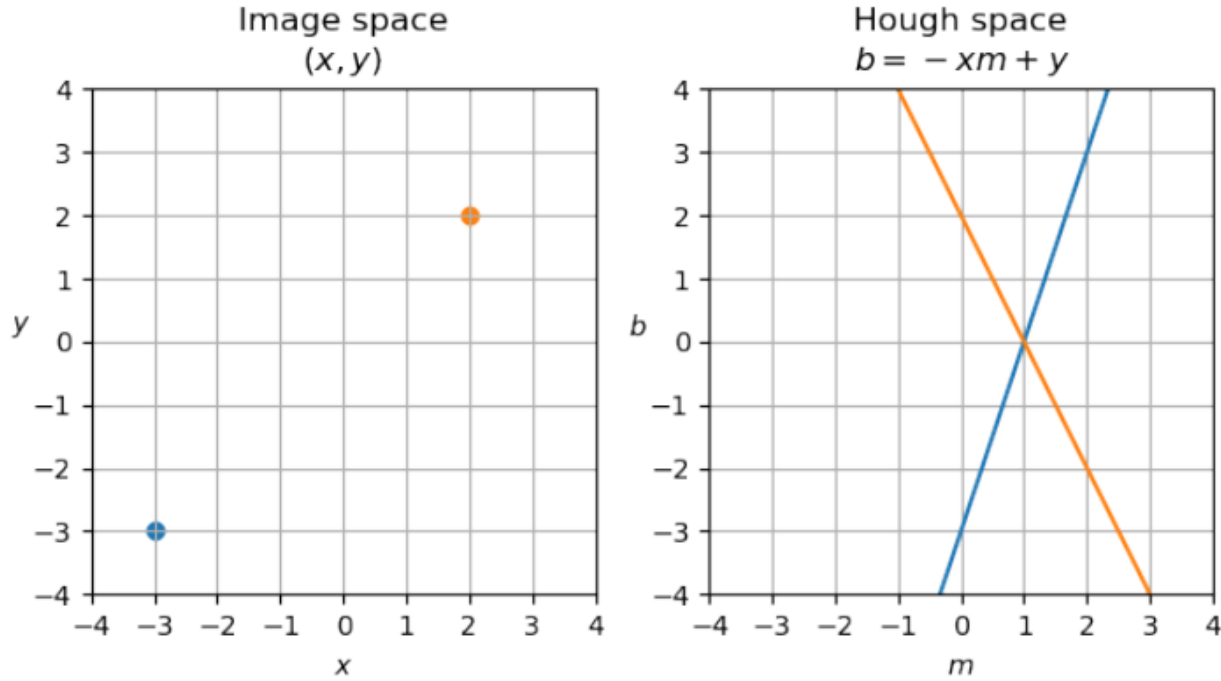
# Hough space

- Each point (edge) in the image space defines 1 line in the Hough space



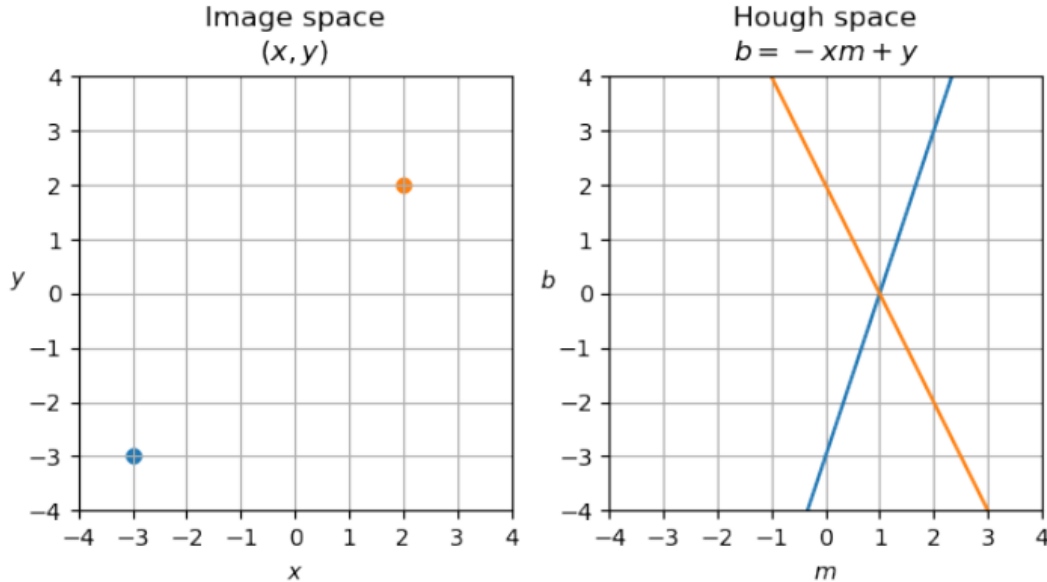
# Hough space

- Each point (edge) in the image space defines 1 line in the Hough space



# Line from Hough

- Can we define 1 line from 2 points?



- Corresponds to which parameters?



## **$y = mx+b$ and vertical lines**

- What are the values of parameters  $\{m,b\}$  for a vertical line?

$$m = \infty, b \in \mathbb{R}$$

- So, this parametrization only works if there are NO vertical lines in the image

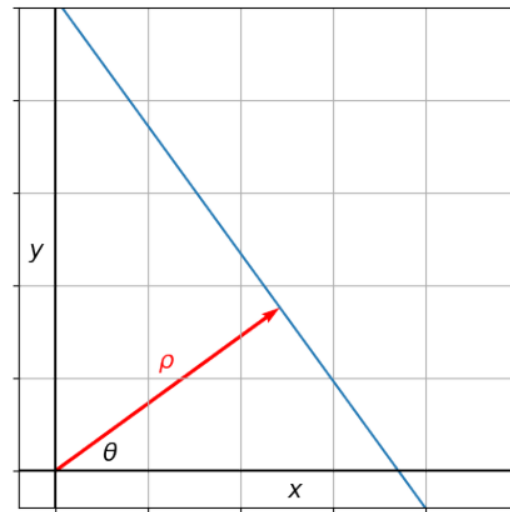
# Parametric polar line model

- By: Duda and Hart, 1972
- Polar representation
- Defines a line by:

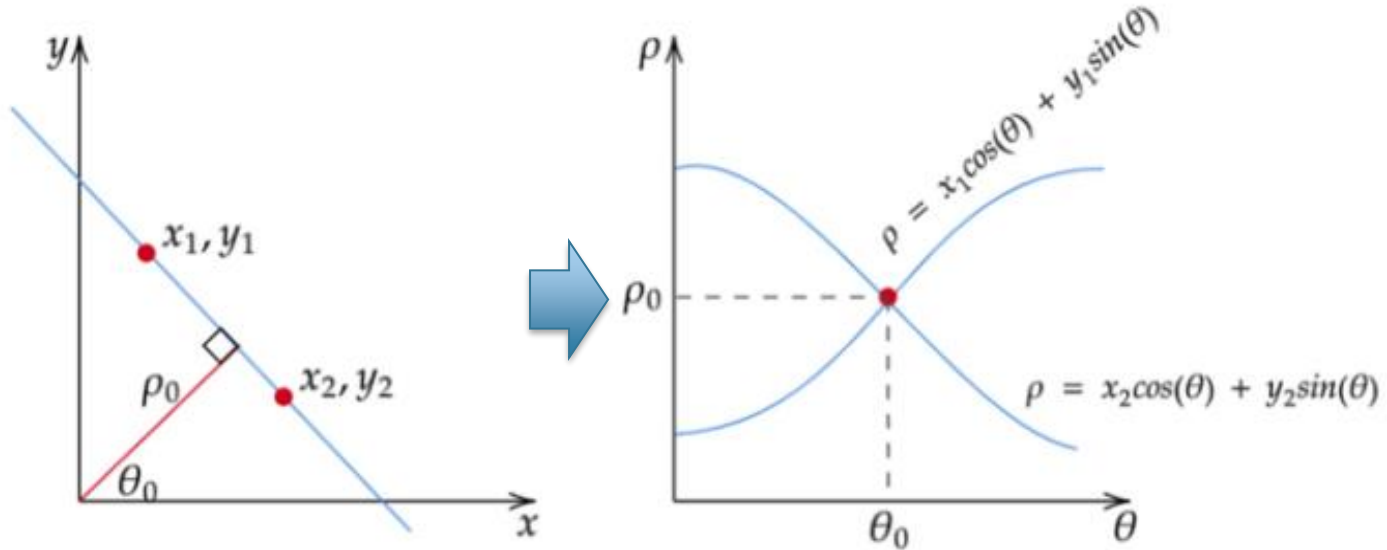
$$y = \frac{\rho - x \cos(\theta)}{\sin(\theta)}$$

- Direction  $\theta$
- Distance  $\rho$  from the origin

In rectangular coordinates

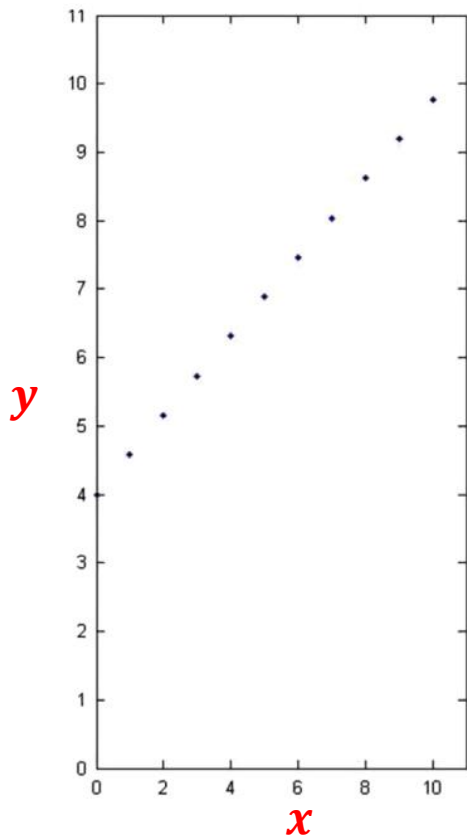


# Polar in Hough space

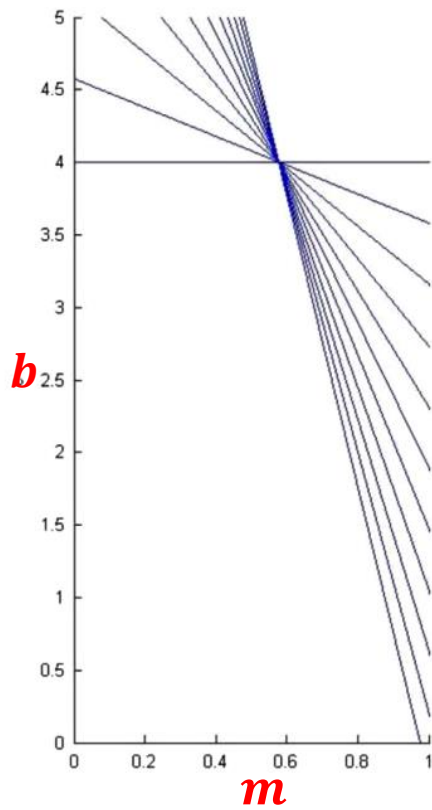


$$y = -\frac{\cos(\theta)}{\sin(\theta)}x + \frac{\rho}{\sin(\theta)} \quad \Rightarrow \quad \rho = x \cos(\theta) + y \sin(\theta)$$

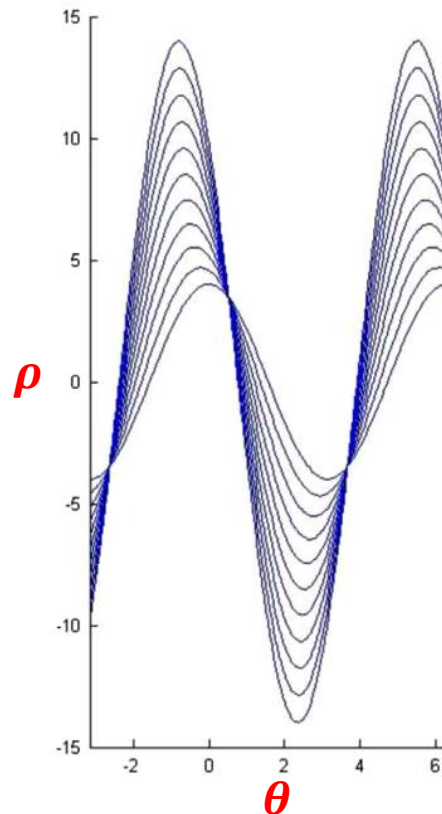
$$y = mx + b$$



$$b = -x m + y$$

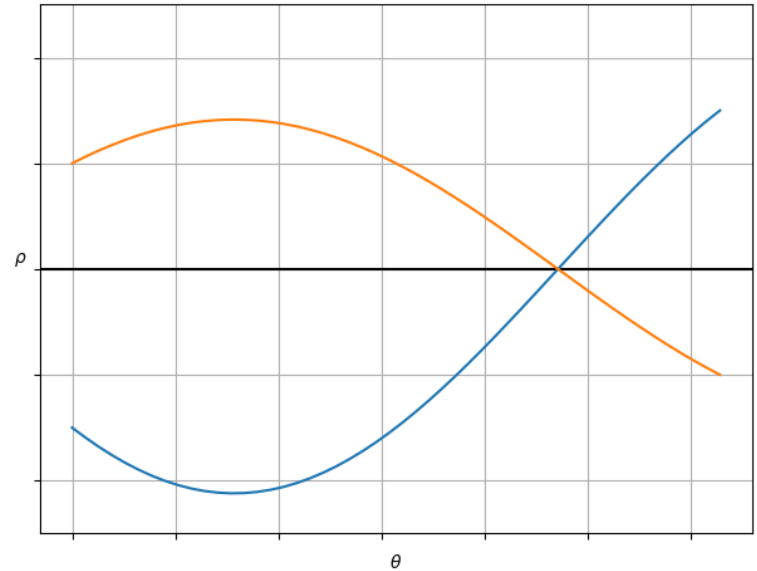


$$\rho = x \cos(\theta) + y \sin(\theta)$$



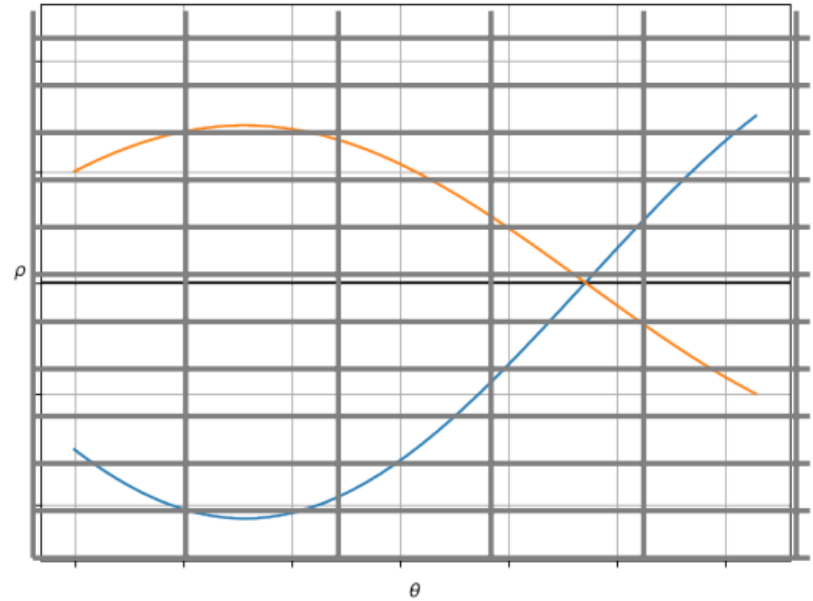
# The accumulator

- We need to count votes
- “Nearby” votes should count as the same vote
- Solution: Discretize the “grid”
- Each cell will be a bin
- $\theta \in [0, \pi]$
- $\rho$  depends on the image size



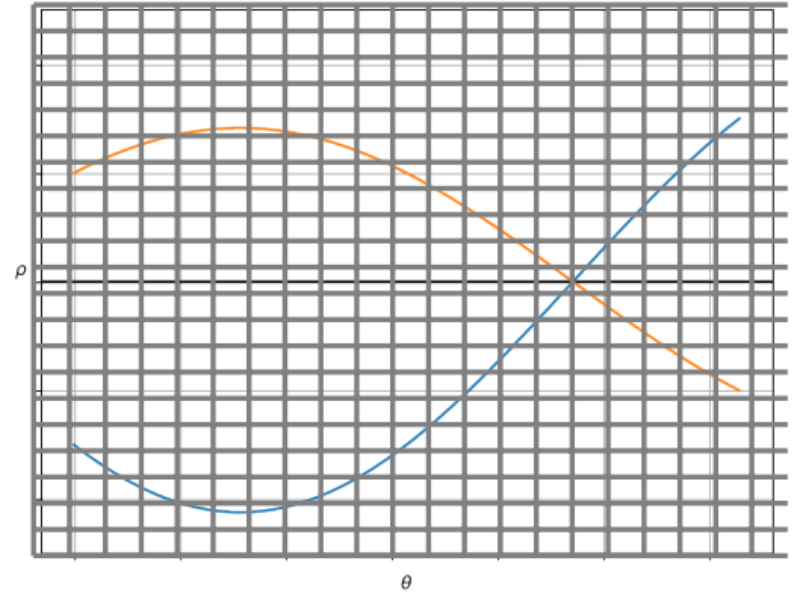
# The accumulator ...

- Note that arrays (i.e. the grid), in programming don't have negative indices
- The level of discretization affects the transform



# The accumulator ...

- Note that arrays (i.e. the grid), in programming don't have negative indices
- The level of discretization affects the transform



# Algorithm

Initialize the accumulator:  $H[\rho, \theta] = 0$

For each edgel in the edge image:

For  $\theta = 0$  to  $2\pi$ :

$$\rho = x \cos(\theta) + y \sin(\theta)$$

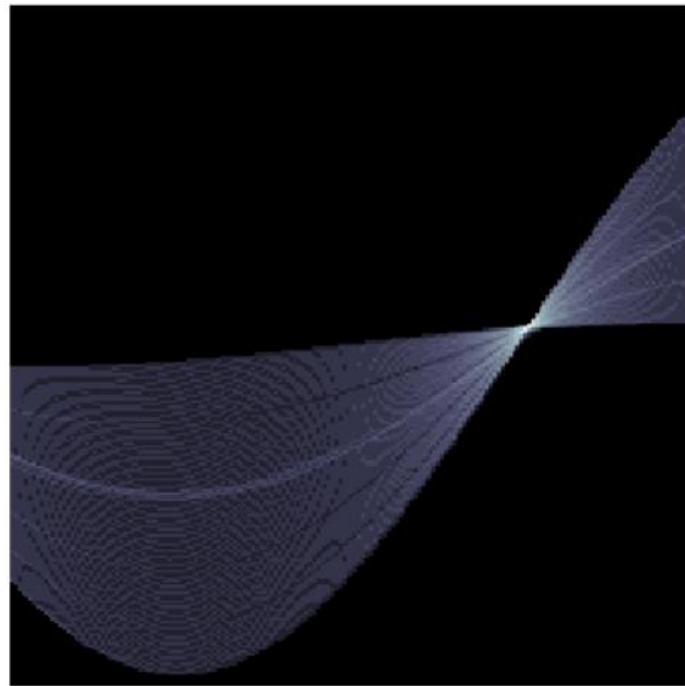
$$H[\rho, \theta] += 1 \quad // \text{note that } \rho \text{ may be negative}$$

Find the '**top**'  $(\rho, \theta)$  pairs

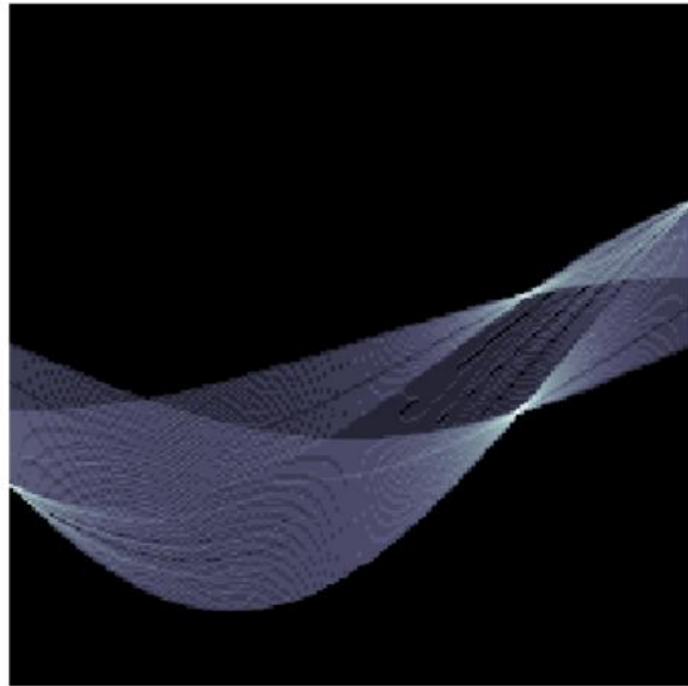
Plot the '**top**' lines



# Hough transform examples



# Hough transform examples



# Recovering lines

- We have  $(\rho, \theta)$  pairs; how we can recover the lines  $(x,y)$ ?

# Recovering lines

- We have  $(\theta, \rho)$  pairs; how we can recover the lines  $(x, y)$ ?
- We can recover the lines with:

$$y = \frac{\rho - x \cos(\theta)}{\sin(\theta)}$$

- Alternatively we can define a line by 2 points:

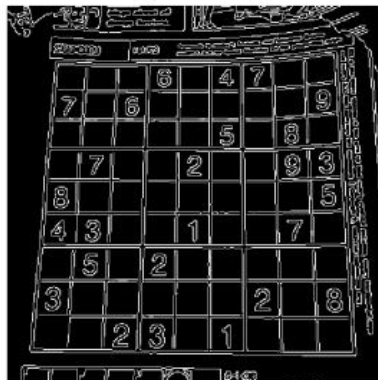
$$\vec{v} = (\rho \cos(\theta), \rho \sin(\theta)) \quad \vec{v}_\perp = (-\rho \sin(\theta), \rho \cos(\theta))$$

$$p_0 = -\infty \vec{v}_\perp + \vec{v} \quad p_1 = \infty \vec{v}_\perp + \vec{v}$$

where  $\infty$  is, in practice replaced by a scalar

# Sudoku

Edge image



Hough space – Polar

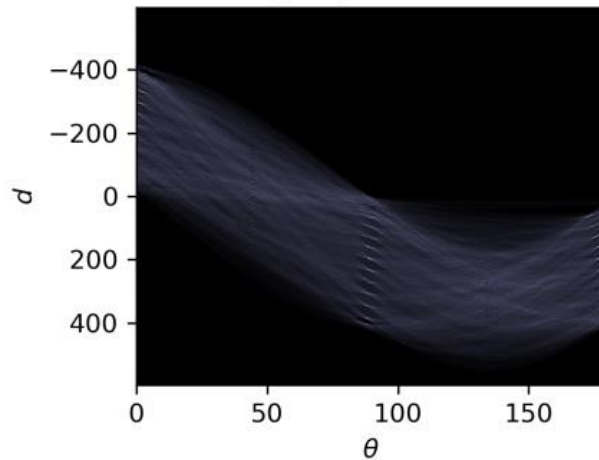
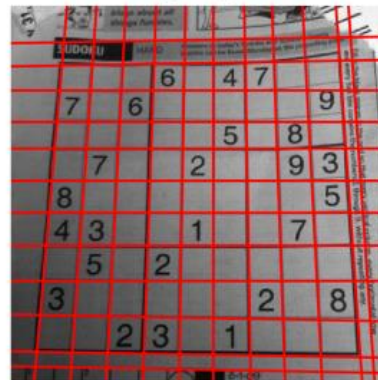
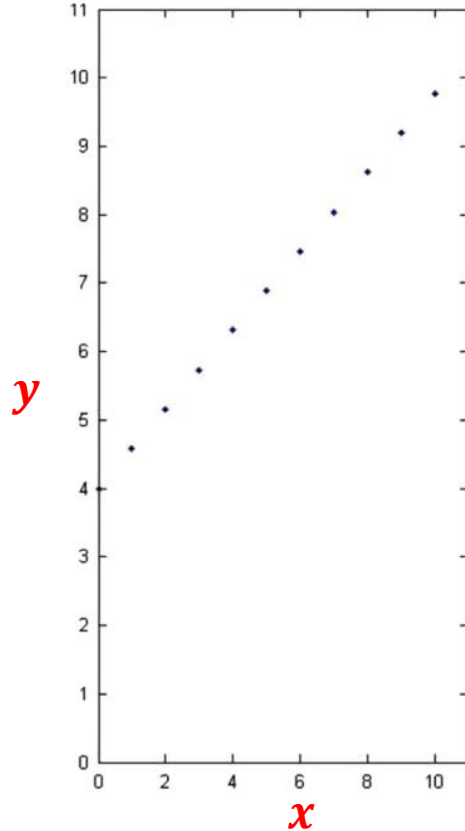


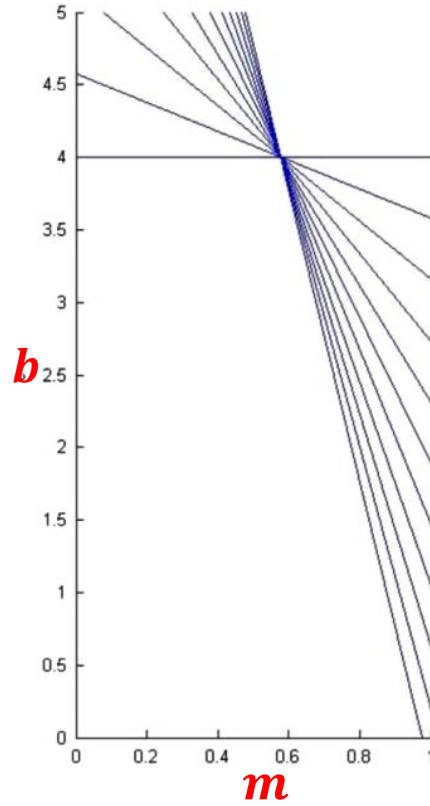
Image space



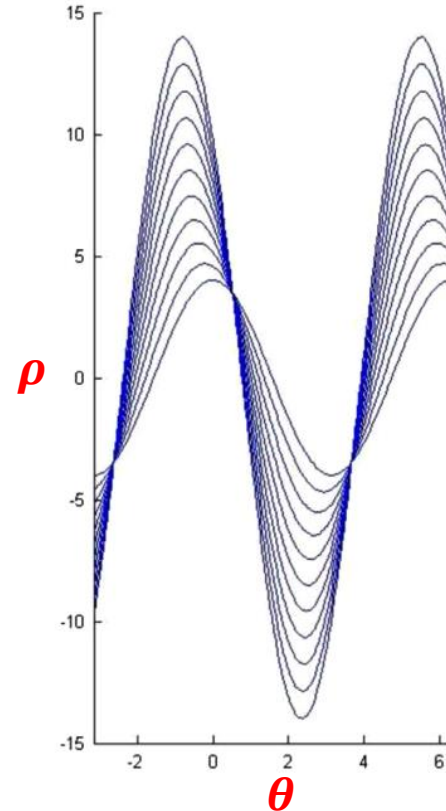
$$y = mx + b$$



$$b = -x m + y$$



$$\rho = x \cos(\theta) + y \sin(\theta)$$



# Back to the banjo man (Canny)



# Back to the banjo man (Hough )







# Hough transform for circles

- The same approach can be used for circle detection
- The equation of a circle:

$$(x_i - x_c)^2 + (y_i - y_c)^2 = r^2$$

{

$(x_c, y_c)$ : centre of the circle

$r$  : radius

- In polar coordinates, it can be expressed in function of the centre:

$$x_c = x_i + r \cos(\theta)$$

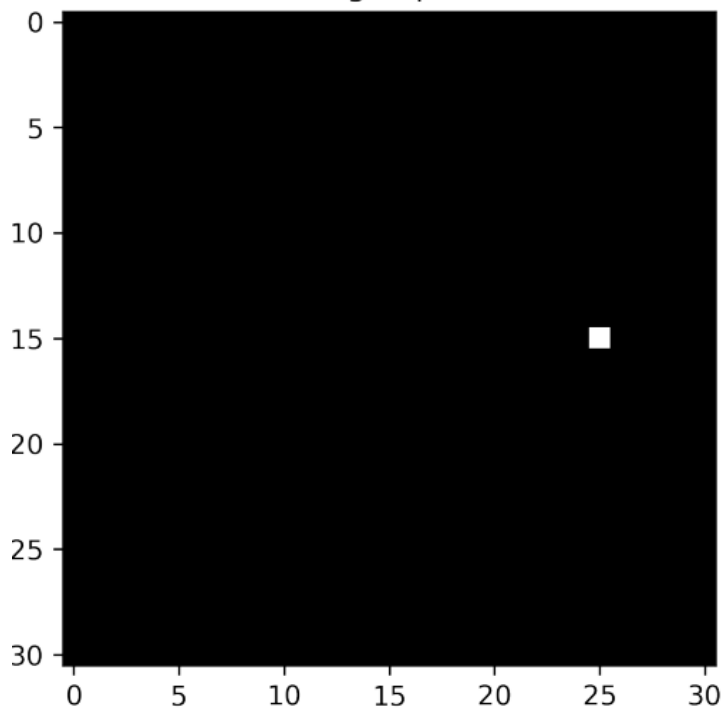
$$y_c = y_i + r \sin(\theta)$$

# Hough circle accumulator (assume known radius)

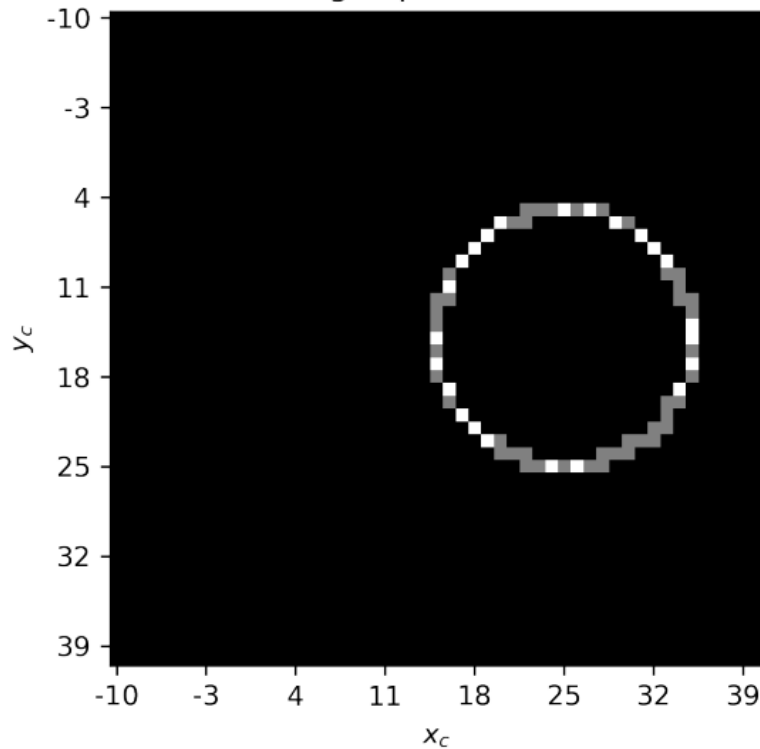
$$x_c = x_i + r \cos(\theta)$$

$$y_c = y_i + r \sin(\theta)$$

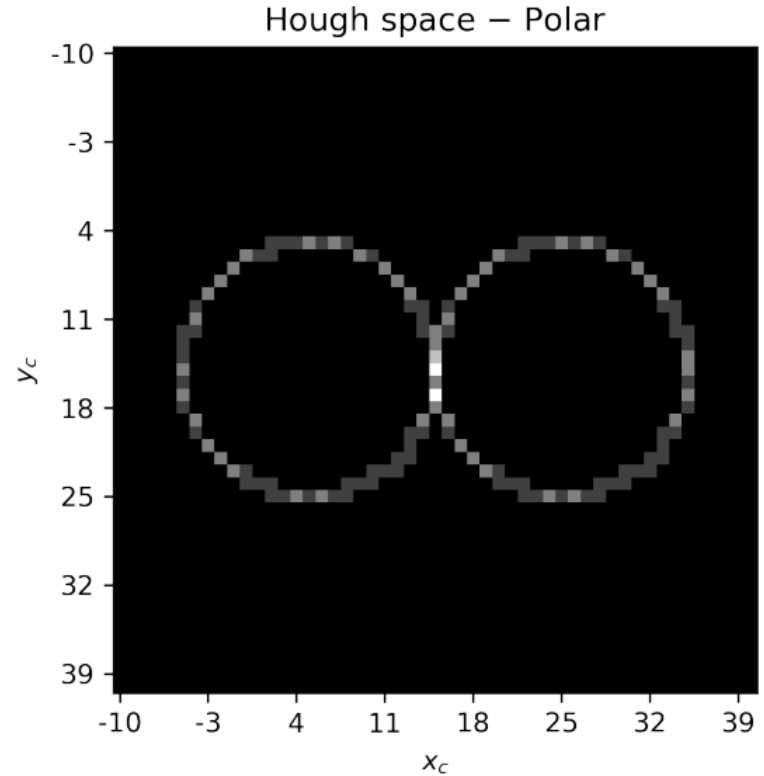
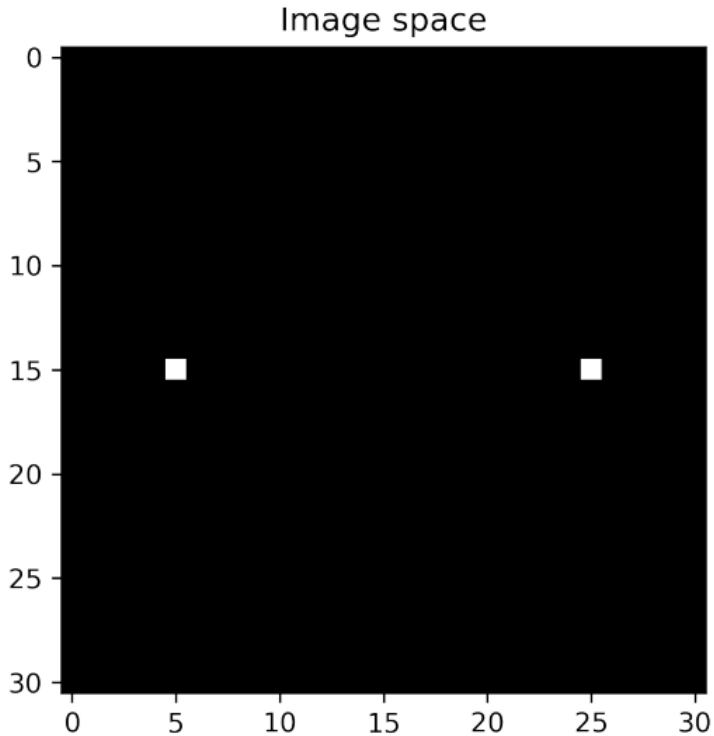
Image space



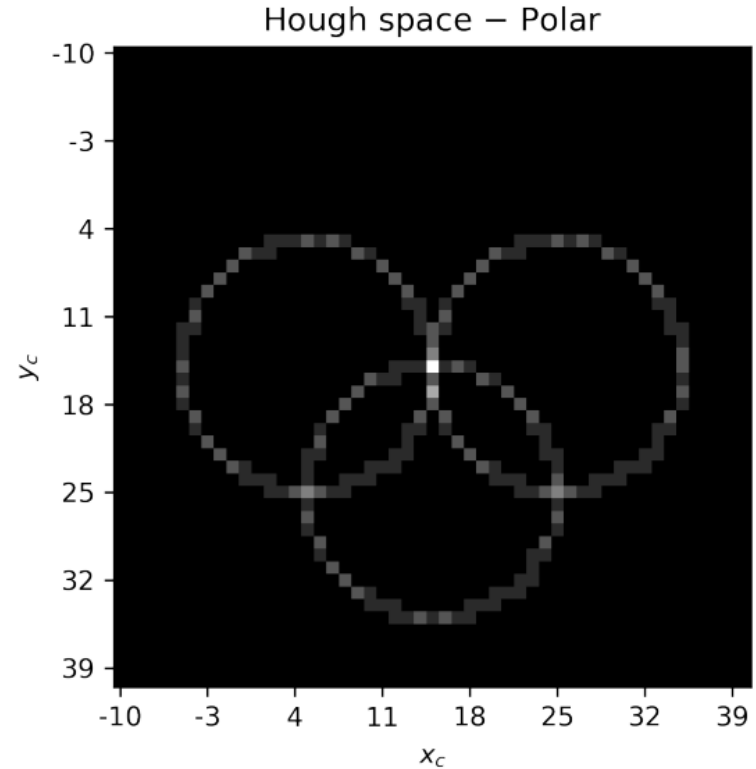
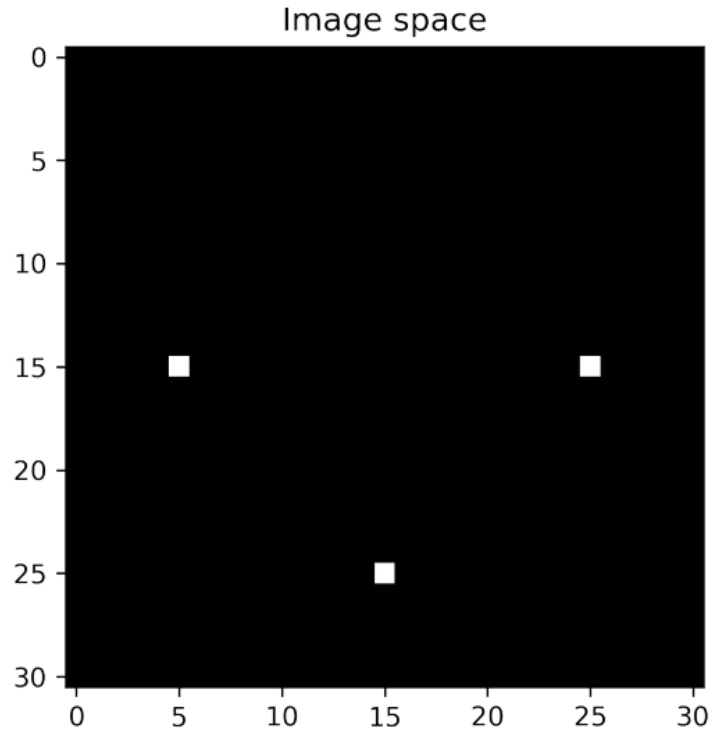
Hough space – Polar



# Hough circle accumulator (assume known radius)



# Hough circle accumulator (assume known radius)



# Algorithm

Initialize the accumulator:  $H[x_c, y_c, r] = 0$

For each edgel  $(x_i, y_i)$  in the edge image:

For every radius  $r$  in a given range:

For  $\theta = 0$  to  $2\pi$ :

$$x_c = x_i + r \cos(\theta)$$

$$y_c = y_i + r \sin(\theta)$$

$$H[x_c, y_c, r] += 1$$

Find the '**top**'  $(x_c, y_c)$  pairs along the radius direction

Plot the '**top**' circles

The image displays a collection of 10 Euro coins, each with a different image processing effect applied to it. The coins are arranged in a grid-like pattern, with some overlapping. The effects include:

- Top-left:** Original color image.
- Top-middle:** Grayscale conversion.
- Top-right:** Edge detection (Canny edges).
- Middle-left:** Thresholding (binary image).
- Middle-middle:** Edge detection (Sobel filter).
- Middle-right:** Edge detection (Laplacian filter).
- Bottom-left:** Edge detection (Prewitt filter).
- Bottom-middle:** Edge detection (Roberts cross).
- Bottom-right:** Edge detection (Scharr filter).

The coins are 10 Euro coins, featuring the Euro symbol and the name 'BERNARDINI'.

# Hough transform: review

- Good:
  - Robust to noise (somehow dependent on bin resolution)
  - Robust to occlusion
- Bad:
  - Computationally intensive
  - Detects even if not present (e.g. lines)
  - Requires decision on bin resolution (not straightforward)



# References

- Szeliski, 4.3.2
- Duda's Hough Transform paper
- Further reference (good survey):

Mukhopadhyay, P., & Chaudhuri, B. B. (2015). A survey of Hough Transform. *Pattern Recognition*, 48(3), 993-1010.